Software Architecture Document

Concurrent File Sharing System (TCP)

Version 1.1

Prepared by [Member names here]

Team 19, Computer Networks Lab

11 November 2026

Revision History

Name	Date	Reason for changes	Version
TEAM 19	11 November 2026	Updated to match repository implementation: POX controller, OVS queues, runtime policies and monitoring	1.1

1. Introduction

1.1 Purpose and scope

This document describes the architecture and design of the Concurrent File Sharing System: a concurrent TCP client-server application with authentication, file listing, upload and download, SHA-256 integrity verification, metadata management and SDN-based network controls. The implementation uses Mininet and Open vSwitch (OVS), with the POX controller framework running a Team 19 custom component. This document maps the repository implementation to the system requirements and records the principal design decisions and limitations.

1.2 Audience

Instructors and evaluators, team members who maintain or extend the code, and testers who need to know where each behaviour lives.

1.3 Definitions

Frame: a 4-byte big-endian length followed by a UTF-8 JSON object. Session: one TCP connection from login to disconnect. OVS: Open vSwitch. POX: Python-based SDN controller framework used to host the custom csfs_pox component. OpenFlow 1.0: southbound protocol used between OVS and POX. OVS queue: host-specific HTB bandwidth queue.

2. Architectural Goals and Constraints

Goal / constraint	Consequence for the design
Correct transfers	Length-prefixed JSON control frames; raw file bytes; SHA-256 verification; temporary upload file and os.replace() before metadata update.
Concurrent sessions	Blocking TCP sockets with one daemon thread per accepted client; metadata and logging locks protect individual shared-state operations.
Small lab deployment	Python implementation with JSON account/metadata files; Mininet topology and one OVS switch.
Role-based access	Server checks file ownership or admin role for listing, upload overwrite and download.
SDN policy enforcement	POX custom component installs OpenFlow 1.0 flows and configures OVS HTB queues for per-host rate limits.
Monitoring and runtime control	Controller polls flow statistics, writes bandwidth_log.csv and accepts status/flows/block/unblock/throttle/unthrottle commands.
Trusted lab environment	TCP is not encrypted; password hashing is salted SHA-256 rather than a password KDF. These are documented security limitations.

3. Architecture Overview

The system combines a concurrent TCP client-server application with an SDN-controlled Mininet network. Multiple clients connect to the server at 10.0.0.100:5000 through OVS switch s1. The server stores accounts, metadata and file contents locally. In SDN mode, s1 connects to POX on 127.0.0.1:6633 using OpenFlow 1.0. The POX framework loads the repository's custom controller component (controller/csfs_pox.py), which installs flow rules, configures OVS HTB queues, receives authentication events from the server and polls flow statistics. The controller's operator interface is provided by controller/controller.py.

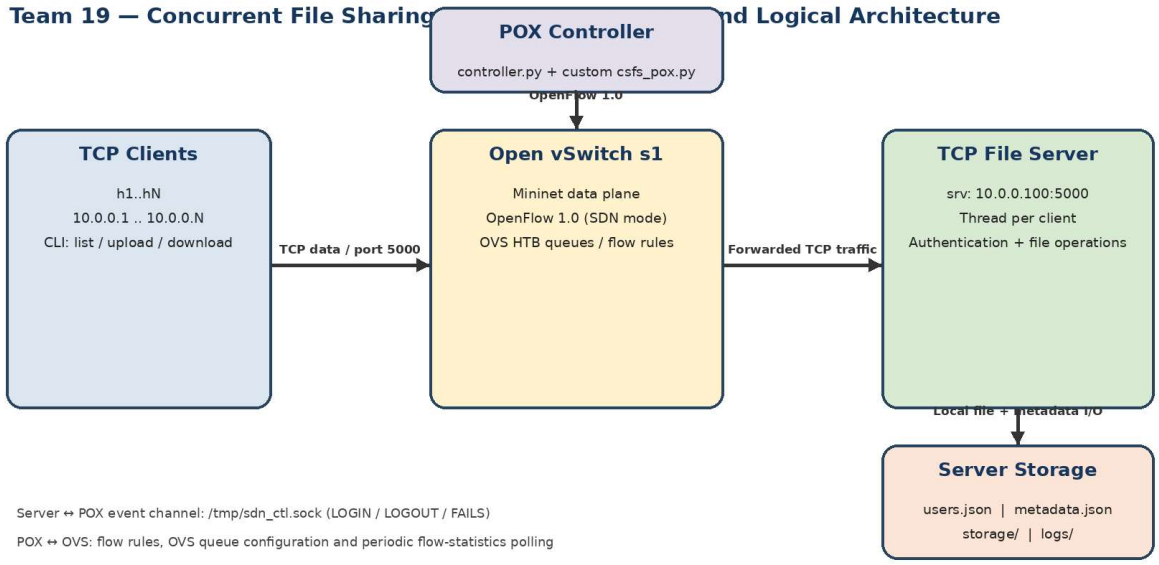


Figure 1: Deployment and logical architecture (controller path detailed in Section 9).

3.1 Components

Component	File / location	Responsibility
TCP server	src/server.py	Accept connections, authenticate and authorise users, list/upload/download files, manage metadata, log activity and notify the controller.
TCP client	src/client.py	CLI and Client class for login, list, upload, download, SHA-256 verification and throughput reporting.
POX launcher / operator CLI	controller/controller.py	Launch POX with csfs_pox component; send operator commands to the controller over the Unix socket.
Custom SDN component	controller/csfs_pox.py	Handle OpenFlow events, install flows, configure queues, process LOGIN/LOGOUT/FAILS events, apply runtime policies and collect flow statistics.
Mininet topology	topology/topo.py	Create server and client hosts, OVS switch s1, OpenFlow 1.0 remote-controller connection in SDN mode, and --nosdn baseline mode.
Account store	users.json	User names, salts, password hashes and roles.
File index	metadata.json	Per-file owner, size, SHA-256 digest and upload timestamp.
File store	storage/	File contents and temporary upload files.
Monitoring output	results/bandwidth_log.csv	Timestamped per-host byte intervals, Mbps, role/state and configured rate.
Logs and tests	logs/; tests/; perf/; results/	Server/controller logs, runtime-policy and hard-scenario tests, performance runners and generated result files.

4. Component Design

4.1 Server

main() creates a TCP listening socket (SO_REUSEADDR, backlog 64) and starts a daemon handler thread for each accepted connection. handle() applies a 120 s socket timeout and runs the framed command loop. Before authentication, only LOGIN is accepted. Three failed attempts on one connection close that connection; a per-IP failure counter also sends a FAILS event to the SDN controller after three failures. Successful login and logout send LOGIN and LOGOUT events through the local Unix socket /tmp/sdn_ctl.sock. Controller notification errors are ignored by the server, so file service can continue even if the controller is unavailable.

Function / module	Purpose
recv_exact / send_msg / recv_msg	Read/write 4-byte length-prefixed JSON frames; server rejects headers larger than 1 MiB.
handle	Per-session authentication, command dispatch, controller LOGIN/LOGOUT/FAILS notifications and cleanup.
do_list / can_access	Return files permitted to the authenticated owner or administrator.
do_upload	Validate name and size (maximum 1 GiB), check overwrite permission, stream in 64 KiB chunks to a random temporary file, verify SHA-256, replace destination and update metadata.
do_download	Check file existence and owner/admin permission, send size and stored digest, then stream file bytes.
safe_name	Use basename and reject empty, '.', '..' and dot-prefixed names.
load_meta / save_meta	Read metadata.json and save through a temporary JSON file followed by os.replace().
notify_ctl / note_fail	Send newline-delimited JSON events to /tmp/sdn_ctl.sock; count failed logins per IP and notify after three failures.
log	Write timestamped server activity to console and logs/server.log under log_lock.

4.2 Client

The Client class opens a TCP connection, authenticates and exposes list(), upload(), download() and close(). Upload computes SHA-256 before sending the UPLOAD header, waits for READY, streams data in chunks up to 64 KiB and reads the final status. Download receives the declared size and digest, streams to the output file while hashing, and reports whether the digest matches. The CLI uses exit codes 0 (success), 1 (operation error), 2 (login failure) and 3 (download integrity failure). Throughput is reported in Mbps.

4.3 SDN controller and policy component

controller/controller.py launches POX with the custom csfs_pox component. The component listens for OpenFlow switch connection events, installs a default-deny base policy with ARP handling, creates per-host OVS HTB queues, and installs TCP flows for traffic between client hosts and server port 5000. It maintains host session, role, policy and byte-counter state. LOGIN assigns the user or administrator rate; LOGOUT returns the host to the unauthenticated rate when its session count reaches zero; FAILS blocks the host for 30 seconds. Operator commands are forwarded over the Unix socket and support status, flows, block, unblock, throttle and unthrottle. The component polls flow statistics and writes sampled bandwidth data to results/bandwidth_log.csv. Rate limits are enforced by OVS HTB queues, not OpenFlow meters.

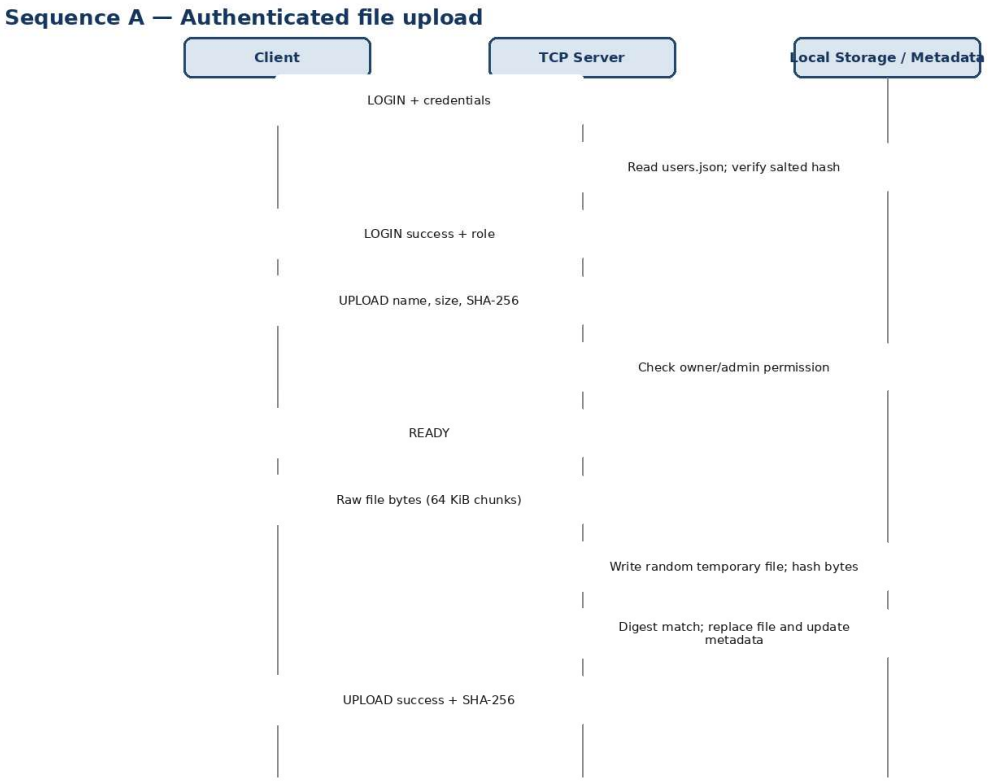
5. Protocol Design

One TCP connection is used per session. Each control message is a 4-byte big-endian length followed by a UTF-8 JSON object; the server rejects control headers larger than 1 MiB. File data follows as raw bytes, outside JSON, and is transferred in chunks of up to 64 KiB. The server accepts files up to 1 GiB.

Step	Client -> Server	Server -> Client
Login	LOGIN {user, password}	{status: ok, role} or {status: error, msg}
List	LIST	{status: ok, files: [...]}
Upload	UPLOAD {name, size, sha256}	{status: ready} or error
	raw bytes (size)	{status: ok, sha256} or {error: integrity check failed}
Download	DOWNLOAD {name}	{status: ok, size, sha256} or error
		raw bytes (size); client verifies hash
End	QUIT	connection closed

Upload sequence: the client calculates SHA-256 and sends the filename, size and digest. The server validates the name and size and checks ownership/admin permission before replying READY. It writes the incoming bytes to a uniquely named temporary file while calculating SHA-256. If the digest matches, the server moves the temporary file into storage with os.replace() and updates metadata.json. On mismatch or interruption, the temporary file is removed and an error is returned or the session ends.

5.1 Upload sequence diagram



6. Concurrency Design

- Model: thread per client. Each accepted TCP connection is handled independently; blocking I/O in one handler does not stall other handlers.
- Shared resources: metadata.json is read or updated under meta_lock at individual metadata operations; server logging is protected by log_lock. users.json is read for authentication. The lock is not held for the entire upload data transfer and final file replacement sequence.
- Per-upload temporary files have random names, so concurrent uploads do not share a temporary path. However, simultaneous uploads using the same final filename are not serialized across the complete transfer/commit operation; this is a known race condition.
- os.replace() atomically replaces a destination path on the same file system. This protects against exposing a partially written temporary file, but it does not by itself make the file replacement and metadata update one transaction. Concurrent overwrite/download operations can therefore race and are listed as a limitation.
- Failure isolation: handled connection, timeout and I/O errors end the affected session; the listening server continues accepting other clients. A controller notification failure is swallowed by notify_ctl(), so application service may continue without updated SDN policy.

Trade-off: thread-per-client is straightforward for the tested Mininet scenarios and the target lab scale. An asyncio or worker-pool design could support different scaling characteristics but was not selected.

7. Data Design

Store / artifact	Structure
users.json	{ name: { salt, hash, role } }; hash is SHA-256(salt + password); roles are user/admin. Default ac alice, bob and admin.
metadata.json	{ filename: { owner, size, sha256, uploaded } }.
storage/	Files named by filename; temporary uploads use random .tmp_ names and are removed after failed/interrupted uploads.
results/bandwidth_log.csv	Controller samples: time, IP, user, policy state, rate_kbps, bytes in interval and Mbps.

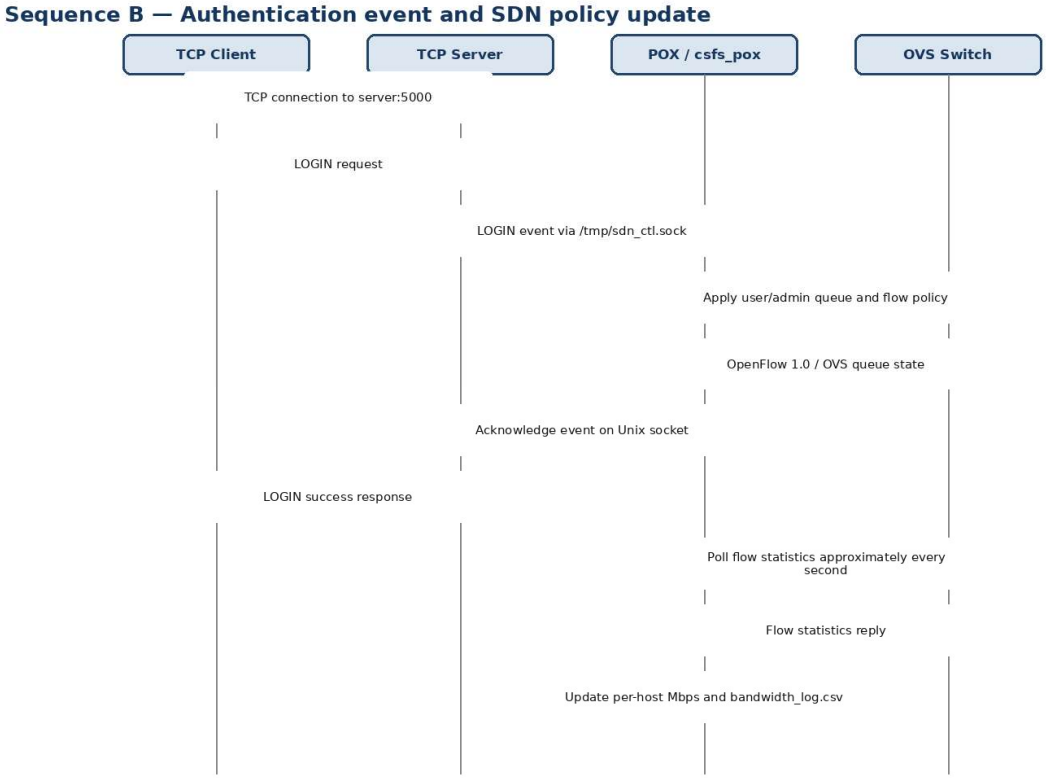
8. Security Design

Concern	Mechanism / limitation
Authentication	Salted SHA-256 hash and constant-time comparison; three failed attempts close one connectio failures are counted and reported to POX after three failures.
Authorisation	Server-side owner/admin checks for LIST, upload overwrite and DOWNLOAD.
Path traversal	safe_name() strips directory components and rejects empty, '.', '..' and dot-prefixed names.
Resource limits	Control header maximum 1 MiB; upload maximum 1 GiB; 120-second connection timeout; upl chunks up to 64 KiB.
Integrity	Server verifies upload SHA-256; client verifies downloaded SHA-256.
Network access policy	POX/OpenFlow 1.0 default-deny flow policy; TCP port 5000 flows and OVS queue rates; failed- and operator controls.
Known security gaps	No TLS; fast SHA-256 password hashing; password supplied as CLI argument; IP-based host ide controller Unix socket chmod 0666. Lab/demo use only; not production-hardened.

9. Network Design (Mininet / OVS)

Mininet creates one OVS switch s1, a server host srv (10.0.0.100/24) and client hosts h1..hN (10.0.0.1/24 onward). In SDN mode, topology/topo.py configures OVS for OpenFlow 1.0, failMode=secure, and a remote POX controller at 127.0.0.1:6633. The controller/controller.py launcher starts POX with the custom csfs_pox component. The component installs a default-deny policy, permits ARP, and installs TCP flows for client/server traffic on port 5000. Per-host rate limits are implemented using OVS HTB queues configured through ovs-vsctl; OpenFlow 1.0 meters are not used. Unauthenticated hosts are limited to 1 Mbps, authenticated users to 10 Mbps, and administrators to 20 Mbps. Runtime controls can block/unblock or throttle/unthrottle a host. Three failed logins trigger a 30-second host block. The controller polls flow statistics approximately once per second and records per-host bandwidth to results/bandwidth_log.csv. The server and controller communicate through newline-delimited JSON events/commands over /tmp/sdn_ctl.sock. The --nosdn topology mode is used as a plain-switch performance baseline; it is not the SDN deployment mode.

9.1 SDN authentication and policy sequence



Controller selection note: The repository uses POX rather than Ryu. This is recorded as the instructor-agreed controller substitution for the project. The implementation is a POX framework instance with a Team 19 custom component; it should not be described as a completely independent controller.

10. Error Handling

Situation	Behaviour
Bad credentials	Error 'invalid credentials'; three failures close the connection; per-IP count sends FAILS event to POX after three failures.
Command before login	Error 'login required'.
Invalid name or size	Error 'invalid name or size'.
Permission problem	Error 'permission denied'; denial is logged.
Unknown file	Error 'file not found'.
Checksum mismatch	Upload rejected, temporary file deleted and integrity failure logged; download client exits with code 3.
Connection lost mid-upload	Temporary file is removed, interruption is logged and session ends; existing destination is not deliberately overwritten until checksum succeeds.
Idle timeout	Connection is closed after 120 seconds.
Controller unavailable	notify_ctl() catches socket errors; TCP file service continues, but SDN authentication/policy updates may not be applied. This degraded mode requires explicit operational testing.
OVS/POX disconnect	Controller removes the Unix socket on shutdown; the switch is configured failMode=secure in SDN mode. Recovery requires restarting/reconnecting the controller and validating flows/queues.

11. Design Decisions

Decision	Alternatives	Reason
Thread per client	asyncio, process pool	Straightforward concurrent handling for the intended lab scale.
Length-prefixed JSON + raw file bytes	Text commands, base64 body	Explicit message boundaries without base64 expansion of file data.
SHA-256 declared before upload	Checksum after upload	Server can verify while receiving without a second file pass.
JSON account and metadata files	SQLite/database	Low setup complexity for the small demonstration dataset.
Temporary file + os.replace()	Write directly to final path	Avoids exposing partially received upload content; does not alone serialize concurrent same-name commits.
POX with custom component	Ryu or manually scripted OVS rules only	POX is the controller framework selected for the agreed lab environment; Team 19 policy logic resides in csfs_pox.py.
OVS HTB queues	OpenFlow 1.0 meters	The topology uses OpenFlow 1.0; rate limiting is implemented with OVS queues configured via ovs-vsctl.

12. Limitations and Future Work

- Add TLS and a slow password hash (bcrypt/scrypt).
- Per-user directories to avoid a global file-name namespace; an admin overwrite currently changes the file's owner.
- Resume of interrupted transfers; delete command; quotas.
- Move metadata to SQLite for larger file counts.

13. Requirements Traceability

SRS feature / requirement	Design element / implementation
1 Authentication	src/server.py handle() LOGIN branch, hash_pw(), users.json; LOGIN event sent to POX.
2 File listing and authorisation	do_list(), can_access(), metadata.json.
3 File upload and integrity	do_upload(), safe_name(), 64 KiB streaming, SHA-256 verification, temporary file and os.replace().
4 File download and integrity	do_download() and Client.download() digest verification.
5 Concurrent sessions	accept loop and thread-per-client handler; individual metadata/log locks.
6 Logging and audit	server log() and logs/server.log; POX policy/controller logs.
7 SDN network policy	topology/topo.py, controller/controller.py and controller/csfs_pox.py; OpenFlow 1.0 flows and OVS HTB queues.
8 Runtime policy management	POX Unix-socket command interface: status, flows, block, unblock, throttle and unthrottle.
9 Bandwidth monitoring	Flow-statistics polling in csfs_pox.py and results/bandwidth_log.csv.